

Design, Analyse and Validate an Efficient Search Strategy for a Large-Scale Student Record System

Design and Analysis of Algorithms — Assignment 1 (CO1)

Student name	D.Hasini
Registration number	192425147
Programme	B.Tech CSE (AI & ML), SIMATS
Random seed (last 4 digits of Reg No)	5147
Algorithms studied	Linear search (iterative) • Binary search (recursive) • Hash table with chaining (third strategy)
Language / tools	Python 3, matplotlib; all timings by <code>time.perf_counter()</code>
GitHub repository	https://github.com/192425147/DAA-A1-192425147-D.Hasini

Contents. Task 1 Design • 2 Basic operation • 3 Time-complexity derivation • 4 Best/average/worst case • 5 Asymptotic analysis • 6 Recurrence formulation • 7 Recurrence solution • 8 Implementation & testing • 9 Experimental performance • 10 Theory vs experiment • 11 Final recommendation • 12 Day-4 design challenge (A-D).

Reproducibility. Every number, table and graph in this report was produced by the code in the repository above (`src/benchmark.py`, `src/task12_benchmark.py`, `src/make_graphs.py`). All screenshots show the registration number, name and system date/time printed by the programs themselves.

Task 1 — Design the Algorithm

Problem. Given a registration number (an integer key) decide whether a student record with that key exists among n records and return its position. The records arrive in no particular order and n grows from 10^3 to 10^6+ . I chose three strategies that rely on three different ideas: *look at everything, repeatedly halve a sorted collection, and compute where the key must be.*

	Algorithm 1	Algorithm 2	Algorithm 3
Name	Linear (sequential) search	Binary search	Hash table with separate chaining
Design strategy	Brute force	Decrease-and-conquer (divide by half, keep one half)	Space-time trade-off (hashing)
Style	Iterative	Recursive	Iterative (build loop + chain scan)
Preprocessing	None — works on raw unsorted records	Sort all records once: $\Theta(n \log n)$	Build the table once: n insertions, $\Theta(n)$ expected
Extra memory	None	Sorted copy (references)	$m \approx n/0.75$ buckets + chains

1.1 Algorithm 1 — Linear search (iterative)

Idea. Inspect the records one after another from the first position and stop as soon as the key matches; if the end is reached the record does not exist. **Why selected:** it is the natural baseline, needs no preprocessing and works on unsorted data, so it shows what every other approach has to beat.

```
LinearSearch(A[0..n-1], key)
  for i <- 0 to n-1 do
    if A[i] = key then          // basic operation
      return i
  return -1                    // not found
```

1.2 Algorithm 2 — Binary search (recursive)

Idea. If the records are sorted, comparing the key with the middle record tells which half can still contain it, so half of the remaining records are discarded after every comparison. **Preprocessing:** the records must first be sorted (once). **Why selected:** it is the standard way to exploit order; its recursive form gives a recurrence relation to analyse in Tasks 6–7. (I also wrote an iterative version of the same strategy in the code, but only to cross-check the recursive one — it is not counted as a separate approach.)

```
Preprocess: S <- sort(A)                // Theta(n log n), done once

BinarySearch(S, key, low, high)
  if low > high then return -1           // base case: empty range
  mid <- floor((low + high) / 2)
  if S[mid] = key then return mid        // basic operation
  else if S[mid] < key then
    return BinarySearch(S, key, mid+1, high)
  else
    return BinarySearch(S, key, low, mid-1)
```

1.3 Algorithm 3 — Hash table with separate chaining

Idea. Instead of searching, compute the position: the hash function $h(k) = k \bmod m$ maps a registration number directly to a bucket, and only the (very short) chain in that bucket is scanned. m is the smallest prime $\geq n/0.75$, so the load factor $\alpha = n/m \leq 0.75$. **Preprocessing:** insert all n records into the table once. **Why selected:** it trades memory and build time for an expected constant search time, which is exactly the trade-off that matters when searches are very frequent (Task 12).

```

Preprocess: m <- smallest prime >= n / 0.75 ; T[0..m-1] <- empty
            for each record at position p with key k: append (k, p) to chain T[k mod m]

HashSearch(T, key)
    chain <- T[key mod m]                // one hash computation
    for each (k, p) in chain do
        if k = key then return p        // basic operation
    return -1

```

Why these three are meaningfully different: brute force with no preprocessing, a comparison-based divide-and-conquer on ordered data, and a direct-addressing approach based on computation instead of comparison. They need different preprocessing (none / sort / build), have different memory needs, and (as shown later) different growth rates.

Task 2 — Identify the Basic Operation

For a search algorithm the dominant work is deciding whether a record is the wanted one, so in all three algorithms the basic operation is the **key comparison** between a stored registration number and the search key. Everything else (index increment, computing mid, computing $k \bmod m$) is done at most a constant number of times per comparison and therefore never dominates; counting comparisons also makes the analysis independent of the machine and programming language.

Algorithm	Basic operation and where it occurs	Why appropriate	How its count changes with n
Linear	$A[i] == \text{key}$ inside the for-loop, once per iteration	The loop body is just this comparison, so loop time $\propto$ comparisons.	Grows in direct proportion to n: up to n comparisons; on average about $n/2$ when the key exists.
Binary (recursive)	$S[\text{mid}] == \text{key}$ once per recursive call	Each call does one comparison and a constant amount of other work, so total time $\propto$ number of calls.	Every comparison halves the range, so the count grows by only 1 when n doubles: about $\log_2 n$.
Hash table	$k == \text{key}$ for each entry of the chain while scanning the bucket	Hashing is one computation; the only repeated work is walking the chain.	Depends on chain length (the load factor α), not on n: about $1 + \alpha/2$ on average, stays near 1.4 for $n = 10^3 \dots 10^6$.

```

# Reg No: 192425147 | Name: D.Hasini
"""Algorithm 1 - Linear (sequential) search. Strategy: brute force, ITERATIVE.
Preprocessing: none. Works on the raw, unsorted records.
Returns (index or -1, number of key comparisons)."""

def linear_search(a, key):
    for i in range(len(a)):
        if a[i] == key:
            return i, i + 1
    return -1, len(a)

# c -- BASIC OPERATION: key comparison
# i: i comparisons were executed
# unsuccessful: all n comparisons executed

```

Figure 2.1: Highlighted basic operation ($A[i] == \text{key}$) in linear search

```

# Reg No: 192425147 | Name: D.Hasini
"""Algorithm 2 - Binary search. Strategy: decrease-and-conquer / divide-and-conquer, RECURSIVE.
Preprocessing: the records must be SORTED once ( $O(n \log n)$ ).
Returns (index or -1, number of key comparisons). One 3-way comparison ( $=, <, >$ ) per call = 1 comparison."""
import time

def preprocess_sort(a):
    """Preprocessing step (included in the efficiency analysis). Returns (sorted_copy, seconds)."""
    t = time.perf_counter()
    s = sorted(a)
    return s, time.perf_counter() - t

def binary_search_recursive(a, key, low=0, high=None):
    if high is None:
        high = len(a) - 1
    if low > high:
        return -1, 0
    mid = (low + high) // 2
    if a[mid] == key:
        return mid, 1
    if a[mid] < key:
        idx, c = binary_search_recursive(a, key, mid + 1, high)
        return idx, c + 1
    else:
        idx, c = binary_search_recursive(a, key, low, mid - 1)
        return idx, c + 1

def binary_search_iterative(a, key):
    """Iterative form of the same strategy (used only to cross-check the recursive one; not a 4th approach)."""
    low, high, c = 0, len(a) - 1, 0
    while low <= high:
        mid = (low + high) // 2
        c += 1
        if a[mid] == key:
            return mid, c
        if a[mid] < key:
            low = mid + 1
        else:
            high = mid - 1
    return -1, c

```

Figure 2.2: Highlighted basic operation ($a[\text{mid}] == \text{key}$) in recursive binary search

```

# Reg No: 192425147 | Name: D.Hasini
"""Algorithm 3 - Hash table with separate chaining. Strategy: space-time trade-off (hashing).
preprocessing: build the table once = n insertions, O(n) expected.
h(k) = k mod m, m = prime >= n / 0.75 (load factor alpha < 0.75).
Returns (record position or -1, number of key comparisons inside the chain)."""
import time

def _is_prime(x):
    if x < 2:
        return False
    i = 2
    while i * i <= x:
        if x % i == 0:
            return False
        i += 1
    return True

def _next_prime(x):
    while not _is_prime(x):
        x += 1
    return x

class HashTable:
    def __init__(self, n_expected, max_load=0.75):
        self.m = _next_prime(int(n_expected / max_load) + 1)
        self.buckets = [None] * self.m
        self.size = 0

    def insert(self, key, pos):
        h = key % self.m          # hash function
        b = self.buckets[h]
        if b is None:
            self.buckets[h] = [(key, pos)]
        else:
            b.append((key, pos))  # chaining (duplicates not expected; reg nos are unique)
            self.size += 1

    def search(self, key):
        b = self.buckets[key % self.m]  # 1 hash computation, O(1) index
        c = 0
        if b is not None:
            for k, pos in b:
                c += 1
                if k == key:          # BASIC OPERATION key comparison inside the chain
                    return pos, c
            return -1, c

```

Figure 2.3: Highlighted basic operation ($k == key$) in the hash-table chain scan

Task 3 — Derive the Time-Complexity Function

Input size: n = number of student records. **Counted operation:** key comparisons $C(n)$. q denotes the number of searches performed.

3.1 Linear search

Worst case. The key is in the last position or absent. The loop runs for $i = 0, 1, \dots, n-1$ and performs one comparison each time: $C_{\text{worst}}(n) = \sum_{i=0}^{n-1} 1 = n$.

Average case (successful). If the key is equally likely to be at any of the n positions, finding it at position i costs i comparisons, so $C_{\text{avg}}(n) = (1/n) \cdot \sum_{i=1}^n i = (1/n) \cdot n(n+1)/2 = (n+1)/2$. If a fraction p of the searches succeed, $C_{\text{avg}} = p(n+1)/2 + (1-p)n$.

Preprocessing: none, so $P(n) = 0$ and the cost of q searches is $q \cdot (n+1)/2$ on average. For $n = 10^6$ this is 500,000.5 comparisons per search.

3.2 Binary search

Worst case. Each call keeps at most $\lfloor n/2 \rfloor$ records, so $C(n) = C(\lfloor n/2 \rfloor) + 1$ with $C(1) = 1$ (derived formally in Tasks 6-7). Unrolling k times until the range is 1 ($n/2^k \approx 1$) gives $C_{\text{worst}}(n) = \lfloor \log_2 n \rfloor + 1$. For $n = 10^6$: $\lfloor 19.93 \rfloor + 1 = 20$.

Average case (successful). Take $n = 2^k - 1$. The binary decision tree has 2^{d-1} keys at depth d ($d = 1 \dots k$), and a key at depth d costs d comparisons: $C_{\text{avg}} = (1/n) \sum_{d=1}^k d \cdot 2^{d-1} = ((k-1)2^k + 1)/n = (k-1) + k/n \approx \log_2(n+1) - 1$. For $n = 10^6$ this is ≈ 18.9 .

Preprocessing. Sorting n keys by a comparison sort needs $\Theta(n \log n)$ comparisons (Python's built-in Timsort, about $n \log_2 n$ for random data), so $P(n) = \Theta(n \log n)$, roughly 2×10^7 comparisons for $n = 10^6$.

Overall cost for q searches: $T(n, q) = n \log_2 n + q \cdot \log_2 n = (n + q) \log_2 n$.

3.3 Hash table with chaining

Assumption: simple uniform hashing — each key is equally likely to land in any of the m buckets. Load factor $\alpha = n/m \leq 0.75$.

Successful search. The i -th inserted key is appended behind the keys already in its bucket; the expected number of earlier keys in that bucket is $(i-1)/m$. Finding it costs $1 + (i-1)/m$ comparisons. Averaging over all n keys: $C_{\text{avg}} = (1/n) \sum_{i=1}^n [1 + (i-1)/m] = 1 + (n-1)/(2m) = 1 + \alpha/2 - \alpha/(2n)$. With $\alpha = 0.75$ this is ≈ 1.375 , independent of n .

Unsuccessful search. The whole chain is scanned: $C = \alpha = 0.75$ comparisons on average (empty buckets cost 0 comparisons) plus one hash computation.

Worst case. If all n keys fall into one bucket the chain has length n and $C_{\text{worst}} = n$ (only for adversarial or badly chosen hash functions).

Preprocessing. n insertions, each one hash computation plus an append: $P(n) = \Theta(n)$ expected.

Overall cost for q searches: $T(n, q) = c_1 n + c_2 q = \Theta(n + q)$.

3.4 Preprocessing and searching together

Algorithm	Preprocessing $P(n)$	One search (average)	Total for q searches	Total when $n = 10^6$, $q = 50,000$ (comparison-scale)
Linear	0	$(n+1)/2$	$q(n+1)/2$	$\approx 25,000,025,000$ comparisons
Binary	$n \log_2 n$	$\log_2(n+1) - 1$	$n \log_2 n + q(\log_2 n)$	$\approx 20,876,569$ comparisons
Hash	n (insertions)	$1 + \alpha/2$	$n + q(1 + \alpha/2)$	$\approx 1,068,750$ inserts+comparisons

Break-even versus linear search. Binary search pays back its sort after q^* searches where $q^* \cdot (n/2 - \log_2 n) \geq n \log_2 n$, i.e. $q^* \approx 2 \log_2 n \approx 40$ searches for $n = 10^6$ (about 20 searches if the saving is counted against the full $2n$ cost of unsuccessful searches). The hash table pays back after

$q^* \approx n/(n/2) = 2$ searches in comparison terms, although its build is the most expensive in real time (see Task 12).

Task 4 — Best, Average and Worst-Case Behaviour

4.1 Linear search

Best case — 1 comparison. The wanted registration number is the first record. The loop exits in its first iteration. **Worst case — n comparisons.** The key is the last record or does not exist (reg no + 1 in my tests), so the loop must look at everything before it can conclude. **Average case — $(n+1)/2$.** When the key is equally likely to be at any position, on average half the list is examined; measured at $n = 10^6$: 502,907 comparisons for existing keys and exactly 1,000,000 for missing keys. The large gap between cases occurs because linear search has no information to skip any record.

4.2 Binary search

Best case — 1 comparison. The key equals the middle record of the whole sorted list (tested with $S[(n-1)/2]$). **Worst case — $\lceil \log_2 n \rceil + 1$ comparisons.** The key is absent (or sits at the deepest level of the decision tree), so the range is halved until it is empty; measured worst values were 10, 14, 17, 20 for $n = 10^3, 10^4, 10^5, 10^6$ — exactly $\lceil \log_2 n \rceil + 1$. **Average case $\approx \log_2(n+1) - 1$.** Half of all keys sit at the last tree level, so the typical search is almost as deep as the worst case: measured 18.88 (found) and 19.95 (not found) at $n = 10^6$. The best and worst cases differ little because every step discards half of the data, whatever the key.

4.3 Hash table

Best case — 1 comparison. The key is the first entry of its chain. **Average case — $1 + \alpha/2 \approx 1.375$** for existing keys and $\alpha = 0.75$ for missing ones; measured 1.38 and 0.77 at $n = 10^6$. A missing key can cost less than one comparison on average because many buckets are empty. **Worst case — the longest chain.** Theoretically $\Theta(n)$ if all keys collide; in my datasets the longest chain was 9 for $n = 10^6$ (and 5, 6, 6 for the smaller sizes), because $k \bmod m$ spreads the random 9-digit numbers well. Collisions, not the data size, create the difference between best and worst.

n	Linear: best / avg(found) / worst	Binary: best / avg(found) / worst	Hash: best / avg(found) / worst
1,000	1 / 500.79 / 1,000	1 / 8.99 / 10	1 / 1.38 / 5
10,000	1 / 4,913.41 / 10,000	1 / 12.38 / 14	1 / 1.34 / 6
100,000	1 / 49,023.29 / 100,000	1 / 15.68 / 17	1 / 1.35 / 6
1,000,000	1 / 502,907.31 / 1,000,000	1 / 18.88 / 20	1 / 1.38 / 9

Table 4.1: Measured number of comparisons per case (from results/results.csv; produced by src/benchmark.py).

Task 5 — Asymptotic Analysis

Algorithm	Searching	Preprocessing	Notation used and why
Linear	Worst/average: $\Theta(n)$; best: $\Theta(1)$. Overall search time is $O(n)$ and $\Omega(1)$	none	$\Theta(n)$ holds for the worst and average cases because they are bounded above and below by cn . The algorithm as a whole is only $O(n)$, since its best case is $\Theta(1)$.
Binary	Worst/average: $\Theta(\log n)$; best: $\Theta(1)$	$\Theta(n \log n)$ sort	Worst case = $\lfloor \log_2 n \rfloor + 1$ is tight from above and below, hence Θ . Total with q searches: $\Theta((n+q) \log n)$.
Hash	Expected $\Theta(1)$ while $\alpha \leq 0.75$; worst $O(n)$	$\Theta(n)$ build	The expected bound is $\Theta(1)$ because $1 + \alpha/2$ lies between 1 and 1.375; the worst case $O(n)$ is only an upper bound reached by total collision.

Growth comparison. Growth rates ordered from slowest to fastest: $1 < \log n < n$, i.e. $\Theta(1) < \Theta(\log n) < \Theta(n)$. Formally, $\lim_{n \rightarrow \infty} \log n / n = 0$ and $\lim_{n \rightarrow \infty} 1/\log n = 0$, so for every constant c the linear cost eventually exceeds $c \cdot \log n$, which in turn exceeds any constant.

n	Linear: $n/2$	Binary: $\log_2 n$	Hash: $1 + \alpha/2$	Linear / Binary
1,000	500	10.0	1.375	50 ×
10,000	5,000	13.3	1.375	376 ×
100,000	50,000	16.6	1.375	3,010 ×
1,000,000	500,000	19.9	1.375	25,086 ×

Table 5.1: Predicted comparisons per successful search.

Interpretation for the university database. When the database grows 1000 times ($10^3 \rightarrow 10^6$ records), linear search becomes 1000 times more expensive ($500 \rightarrow 500,000$ comparisons), binary search only about twice as expensive ($10 \rightarrow 20$ comparisons, one extra comparison every time the data doubles), and the hash table does not get more expensive in comparisons at all. The measured wall-clock times follow the same pattern: linear grew 1,242×, binary 6.1× and hash 9.9× (Task 10 explains why hash time is not perfectly flat). For preprocessing the order is reversed: none < sort ($n \log n$) < hash build in real time, so the benefit must be earned back by enough searches (Task 12).

Task 6 — Formulate the Recurrence Relation

The recursive algorithm is binary search. I derive the recurrence directly from the code in Task 1.

```

BinarySearch(S, key, low, high)
  if low > high then return -1           // (1) base case
  mid <- floor((low+high)/2)
  if S[mid] = key then return mid        // (2) one comparison + 0(1) work
  else if S[mid] < key then return BinarySearch(S, key, mid+1, high) // (3) ONE call on right part
  else return BinarySearch(S, key, low, mid-1) // (3) ONE call on left part

```

Component	Identification from the algorithm
Input size of the original problem	$n = \text{high} - \text{low} + 1$, the number of records still under consideration (initially all n records)
Size of the recursive subproblem	The call is made either on $S[\text{mid}+1..\text{high}]$ (size $\text{high} - \text{mid} = \lfloor n/2 \rfloor$) or on $S[\text{low}..\text{mid}-1]$ (size $\text{mid} - \text{low} = \lfloor (n-1)/2 \rfloor$). In the worst case the larger part, of size $\lfloor n/2 \rfloor$, is taken.
Number of recursive calls	One per invocation (only one of the two branches is executed), not two — the other half is discarded.
Work outside the recursive call	Computing mid, one comparison $S[\text{mid}] = \text{key}$ and one $S[\text{mid}] < \text{key}$ test, all constant time: c operations = $\Theta(1)$ (counted as 1 key comparison).
Base case	$n = 0$ ($\text{low} > \text{high}$): return -1 without comparison, $T(0) = 0$ (or 1 operation if counting the test). A successful match gives $n = 1 \rightarrow$ one comparison: $T(1) = 1$.

Putting the pieces together. The cost of searching n records equals the cost of the constant work in this call plus the cost of searching at most $\lfloor n/2 \rfloor$ records:

$T(n) = T(\lfloor n/2 \rfloor) + c$, $n > 1$, $T(1) = c_0$ (constant). Counting comparisons only ($c = 1$, $c_0 = 1$):
 $C(n) = C(\lfloor n/2 \rfloor) + 1$, $C(1) = 1$.

Check on a small case: $n = 7 \rightarrow$ the call compares the middle element and recurses on at most 3 elements $\rightarrow$ at most 1 element $\rightarrow$ total at most $1 + 1 + 1 = 3$ comparisons = $\lfloor \log_2 7 \rfloor + 1$. The code's own counter confirms this (maximum 3 for $n = 7$).

Task 7 — Solve the Recurrence Relation

Recurrence from Task 6: $T(n) = T(\lfloor n/2 \rfloor) + c$, $T(1) = c_0$. I solve it with the **iteration (back-substitution) method** and then confirm it with the **Master Theorem** and by **induction (substitution)**.

7.1 Iteration method

Why applicable: the recurrence has a single recursive term whose argument is halved each time, so repeated expansion gives a simple pattern. For $n = 2^k$ ($\lfloor \cdot \rfloor$ vanishes):

$$\begin{aligned} T(n) &= T(n/2) + c \\ &= T(n/4) + c + c = T(n/4) + 2c \\ &= T(n/8) + 3c \\ &\vdots \\ &= T(n/2^i) + i \cdot c \quad (\text{pattern after } i \text{ expansions}) \end{aligned}$$

Stop when $n/2^i = 1 \rightarrow i = \log_2 n$:
 $T(n) = T(1) + c \cdot \log_2 n = c_0 + c \cdot \log_2 n \in \Theta(\log n)$

For general n the floors only change the count by at most one level, giving $T(n) = c(\lfloor \log_2 n \rfloor + 1) + c_0 - c$, still $\Theta(\log n)$.

7.2 Master Theorem check

The Master Theorem applies to $T(n) = aT(n/b) + f(n)$ with $a \geq 1$, $b > 1$. Here $a = 1$, $b = 2$, $f(n) = c = \Theta(1)$. Compute $n^{\log_b a} = n^{\log_2 1} = n^0 = 1$. Since $f(n) = \Theta(n^{\log_b a}) = \Theta(1)$, **Case 2** applies: $T(n) = \Theta(n^{\log_b a} \log n) = \Theta(\log n)$. Both methods agree.

7.3 Substitution (induction) proof

Claim: $T(n) \leq c \log_2 n + c_0$ for $n \geq 1$. **Base:** $T(1) = c_0 = c \cdot 0 + c_0$. **Step:** assume it for $n/2$. Then $T(n) = T(n/2) + c \leq c \log_2(n/2) + c_0 + c = c(\log_2 n - 1) + c_0 + c = c \log_2 n + c_0$. ■

7.4 What the result tells us

$T(n) = \Theta(\log n)$ means the recursion depth — and therefore the number of comparisons — grows only by a constant every time n doubles: about 10 levels for 1,000 records, 20 for 1,000,000 and just 30 for 10^9 records. The recursion also never grows larger than the $O(\log n)$ call stack, so the recursive implementation is safe even for millions of records (maximum depth 20 here). The constant work per level is what keeps the cost small; if the algorithm had made *two* recursive calls ($a = 2$) the same recurrence would give $\Theta(n)$ and lose the entire benefit.

Task 8 — Implementation and Testing

Implementation. All three algorithms are implemented in Python exactly as written in Task 1 (src/algo1_linear_search.py, algo2_binary_search.py, algo3_hash_table.py). Each search function returns the position (or -1) and the number of key comparisons it executed, so the same run gives correctness evidence and operation counts.

Datasets. src/dataset_generator.py creates $n = 1,000, 10,000, 100,000$ and $1,000,000$ unique 9-digit registration numbers in random order, using the random seed 5147 (last four digits of 192425147). Every dataset contains my own registration number 192425147 (found test) and, by construction, does **not** contain 192425148 (not-found test). The three algorithms always receive the same records: linear search the raw list, binary search a sorted copy, and the hash table is built from the raw list.

Test procedure (src/test_correctness.py) — for each size, 26 existing keys (own reg no, first and last record of the raw list, smallest, largest and median of the sorted list, 20 random keys) and 6 non-existing keys (reg no + 1, 0, smaller than minimum, larger than maximum, 999,999,999, median + 1), plus edge cases (empty list and single record). Each result is checked against an independent oracle (a Python dictionary of the records), and the iterative binary search is used to cross-check the recursive one.

Test case	Expected result	Why included
Own reg no 192425147	found (all three algorithms, correct record)	personal key required by the rules
Own reg no + 1 = 192425148	not found	personal key required by the rules
First / last record (raw), min / max (sorted), median	found	boundary positions and the binary-search best case
0, below min, above max, 999,999,999	not found	out-of-range keys
20 random existing keys	found	general behaviour
empty list; one-record list	not found / found	degenerate sizes

```
Command Prompt
The system cannot find the path specified.

C:\Users\hasin>"C:\Users\hasin\OneDrive\Desktop\DAA-A1-192425147-D.Hasini"
"C:\Users\hasin\OneDrive\Desktop\DAA-A1-192425147-D.Hasini" is not recognized as an internal or external command,
operable program or batch file.

C:\Users\hasin>cd C:\Users\hasin\OneDrive\Desktop\DAA-A1-192425147-D.Hasini\src
C:\Users\hasin\OneDrive\Desktop\DAA-A1-192425147-D.Hasini\src>dir
Volume in drive C is Windows
Volume Serial Number is CA9B-F22D

Directory of C:\Users\hasin\OneDrive\Desktop\DAA-A1-192425147-D.Hasini\src
06-10-2026 14:45 <DIR>          .
06-10-2026 14:45 <DIR>          ..
06-10-2026 14:45              466 algo1_linear_search.py
06-10-2026 14:45              1,476 algo2_binary_search.py
06-10-2026 14:45              1,680 algo3_hash_table.py
06-10-2026 14:45              5,660 benchmark.py
06-10-2026 14:45              538 common.py
06-10-2026 14:45              1,837 dataset_generator.py
06-10-2026 14:45              5,173 make_graphs.py
06-10-2026 14:45              6,152 task12_benchmark.py
06-10-2026 14:45              3,291 test_correctness.py
06-10-2026 14:45              9 File(s)          26,273 bytes
06-10-2026 14:45              2 Dir(s)          334,142,789,760 bytes free

C:\Users\hasin\OneDrive\Desktop\DAA-A1-192425147-D.Hasini\src>python test_correctness.py
Reg No: 192425147 | Name: D.Hasini | Seed: 5147 | 2026-10-06 14:40:18
TASK 8 - Correctness tests: Linear | Binary (recursive + iterative check) | Hash table
=====
edge cases (empty list, single element) -> ALL PASS
n = 1,000 | test keys = 32 (found 26, not found 6) | own reg no found: True | reg no+1 found: False | checks 128 -> ALL PASS
n = 10,000 | test keys = 32 (found 26, not found 6) | own reg no found: True | reg no+1 found: False | checks 128 -> ALL PASS
n = 100,000 | test keys = 32 (found 26, not found 6) | own reg no found: True | reg no+1 found: False | checks 128 -> ALL PASS
n = 1,000,000 | test keys = 32 (found 26, not found 6) | own reg no found: True | reg no+1 found: False | checks 128 -> ALL PASS
=====
TOTAL checks: 517 | passed: 517 | failed: 0
RESULT: ALL ALGORITHMS CORRECT
```

Figure 8.1: Correctness test output for all algorithms at $n = 1,000, 10,000, 100,000$ and $1,000,000$ (517 checks, 0 failures)

Outcome: all 517 checks passed for all four dataset sizes, so the three implementations are correct and are the programs analysed in Tasks 1-7. **How to reproduce:** `cd src && python test_correctness.py` (Python 3.8+, no external libraries).

Task 9 — Record Experimental Performance

Method. src/benchmark.py measures, for each size, the preprocessing time and the average time and number of comparisons per search. The same seeded random sample of existing keys (the first one is my own reg no) and non-existing keys (the first one is reg no + 1) is used for every algorithm. Binary search and hashing use 1,000 found and 1,000 not-found keys; linear search uses 1,000 keys up to $n = 10^5$ and 100 keys at $n = 10^6$ (otherwise one run would take minutes). Time is the best of 3 repeats for $n \leq 10^4$ and a single pass for larger n , measured with `time.perf_counter()`; the same machine and the same Python process settings were used for every row.

Algorithm	Preprocessing (ms)	Avg time, found (μ s)	Avg time, not found (μ s)	Avg comps, found	Avg comps, not found	Best	Worst
Linear search	0.00	9.51	18.95	500.79	1,000	1	1,000
Binary search (recursive)	0.07	1.08	1.25	8.99	9.98	1	10
Hash table (chaining)	0.33	0.15	0.14	1.38	0.71	1	5

Table 9.1: Results for $n = 1,000$ (queries: linear 1000, binary/hash 1000 found keys).

Algorithm	Preprocessing (ms)	Avg time, found (μ s)	Avg time, not found (μ s)	Avg comps, found	Avg comps, not found	Best	Worst
Linear search	0.00	102.63	220.62	4,913	10,000	1	10,000
Binary search (recursive)	0.95	1.66	1.88	12.38	13.37	1	14
Hash table (chaining)	3.03	0.16	0.14	1.34	0.70	1	6

Table 9.2: Results for $n = 10,000$ (queries: linear 1000, binary/hash 1000 found keys).

Algorithm	Preprocessing (ms)	Avg time, found (μ s)	Avg time, not found (μ s)	Avg comps, found	Avg comps, not found	Best	Worst
Linear search	0.00	1,278	2,356	49,023	100,000	1	100,000
Binary search (recursive)	15.94	2.56	3.16	15.68	16.69	1	17
Hash table (chaining)	92.57	1.22	0.94	1.35	0.72	1	6

Table 9.3: Results for $n = 100,000$ (queries: linear 1000, binary/hash 1000 found keys).

Algorithm	Preprocessing (ms)	Avg time, found (μ s)	Avg time, not found (μ s)	Avg comps, found	Avg comps, not found	Best	Worst
Linear search	0.00	11,816	26,946	502,907	1,000,000	1	1,000,000
Binary search (recursive)	258.49	6.61	6.53	18.88	19.95	1	20
Hash table (chaining)	2,084	1.52	0.76	1.38	0.77	1	9

Table 9.4: Results for $n = 1,000,000$ (queries: linear 100, binary/hash 1000 found keys).

```

Command Prompt
C:\Users\hasin\OneDrive\Desktop\DAA-A1-192425147-D.Hasini\src>python benchmark.py --n 1000
Reg No: 192425147 | Name: D.Hasini | Seed: 5147 | 2026-10-06 14:54:26
TASK 9 - Benchmark: Linear vs Binary (recursive) vs Hash table
=====
Dataset size n = 1,000 (seed 5147; own reg no 192425147 = found, 192425148 = not found)
Algorithm      prep(ms)  us/search  comps(found)  comps(miss)  best  worst
-----
Linear search   0.00      9.51       500.79        1000.00       1    1000
Binary search (recursive) 0.07      1.08        8.99          9.98         1     10
Hash table (chaining) 0.33      0.15        1.38          0.71         1     5
(hash table: m = 1,361 buckets, load factor = 0.735; timings = mean per search; queries: linear 1000, binary/hash 1000)

results saved to results/results.csv

C:\Users\hasin\OneDrive\Desktop\DAA-A1-192425147-D.Hasini\src>

```

Figure 9.1: Benchmark terminal output for $n = 1,000$

```

Command Prompt
C:\Users\hasin\OneDrive\Desktop\DAA-A1-192425147-D.Hasini\src>python benchmark.py --n 10000
Reg No: 192425147 | Name: D.Hasini | Seed: 5147 | 2026-10-06 14:55:20
TASK 9 - Benchmark: Linear vs Binary (recursive) vs Hash table
=====
Dataset size n = 10,000 (seed 5147; own reg no 192425147 = found, 192425148 = not found)
Algorithm      prep(ms)  us/search  comps(found)  comps(miss)  best  worst
-----
Linear search   0.00     102.63    4913.41      10000.00       1  10000
Binary search (recursive) 0.95      1.66      12.38        13.37         1    14
Hash table (chaining) 3.03      0.16       1.34         0.70         1     6
(hash table: m = 13,337 buckets, load factor = 0.750; timings = mean per search; queries: linear 1000, binary/hash 1000)

results saved to results/results.csv

C:\Users\hasin\OneDrive\Desktop\DAA-A1-192425147-D.Hasini\src>

```

Figure 9.2: Benchmark terminal output for $n = 10,000$

```

C:\Users\hasin\OneDrive\Desktop\DAA-A1-192425147-D.Hasini\src>python benchmark.py --n 100000
Reg No: 192425147 | Name: D.Hasini | Seed: 5147 | 2026-10-06 22:55:55
TASK 9 - Benchmark: Linear vs Binary (recursive) vs Hash table
=====
Dataset size n = 100,000 (seed 5147; own reg no 192425147 = found, 192425148 = not found)
Algorithm      prep(ms)  us/search  comps(found)  comps(miss)  best  worst
-----
Linear search   0.00      1278.20     49023.29      100000.00    1    100000
Binary search (recursive) 15.94      2.56       15.68         16.69        1     17
Hash table (chaining) 92.57      1.22        1.35          0.72         1     6
(hash table: m = 133,337 buckets, load factor = 0.750; timings = mean per search; queries: linear 1000, binary/hash 1000)

results saved to results/results.csv

C:\Users\hasin\OneDrive\Desktop\DAA-A1-192425147-D.Hasini\src>

```

Figure 9.3: Benchmark terminal output for $n = 100,000$

```

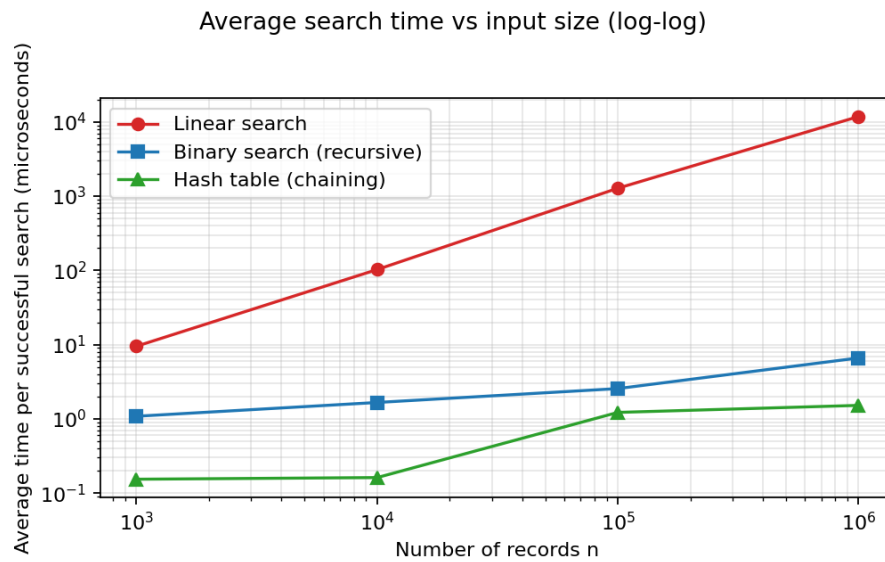
C:\Users\hasin\OneDrive\Desktop\DAA-A1-192425147-D.Hasini\src>python benchmark.py --n 1000000
Reg No: 192425147 | Name: D.Hasini | Seed: 5147 | 2026-10-06 22:56:38
TASK 9 - Benchmark: Linear vs Binary (recursive) vs Hash table
=====
Dataset size n = 1,000,000 (seed 5147; own reg no 192425147 = found, 192425148 = not found)
Algorithm      prep(ms)  us/search  comps(found)  comps(miss)  best  worst
-----
Linear search   0.00     11816.49    502907.31     1000000.00    1    1000000
Binary search (recursive) 258.49      6.61       18.88         19.95        1     20
Hash table (chaining) 2083.93     1.52        1.38          0.77         1     9
(hash table: m = 1,333,357 buckets, load factor = 0.750; timings = mean per search; queries: linear 100, binary/hash 1000)

results saved to results/results.csv

C:\Users\hasin\OneDrive\Desktop\DAA-A1-192425147-D.Hasini\src>

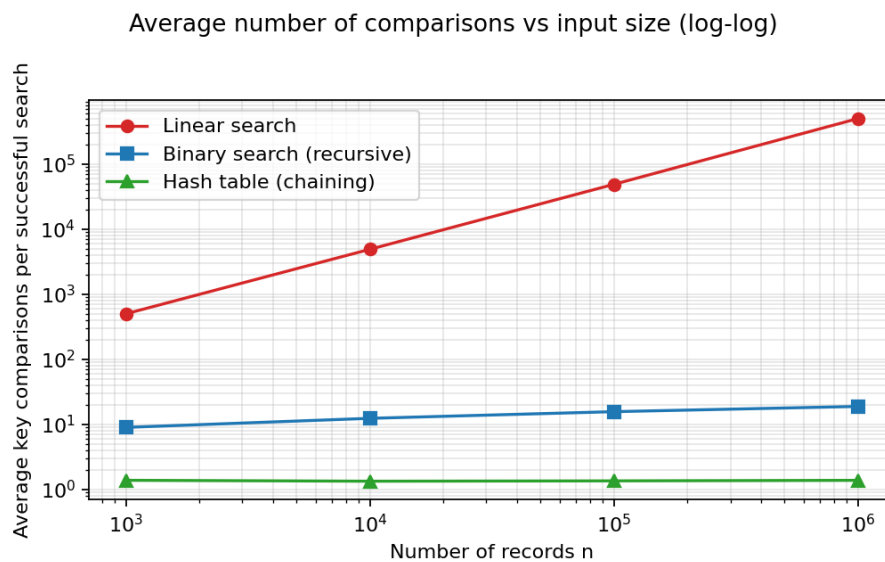
```

Figure 9.4: Benchmark terminal output for $n = 1,000,000$



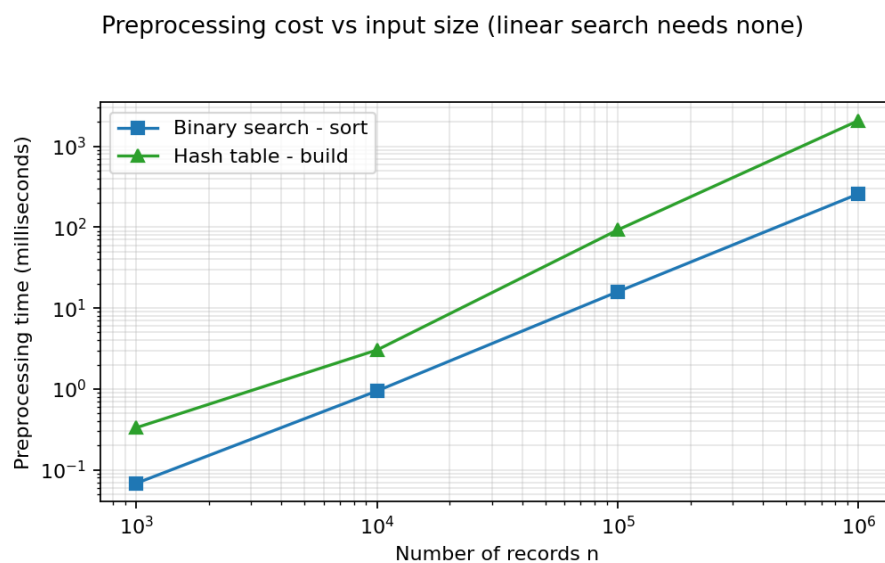
Reg No: 192425147 | D.Hasini

Figure 9.5: Average search time vs n (log-log axes)



Reg No: 192425147 | D.Hasini

Figure 9.6: Average number of comparisons vs n (log-log axes)



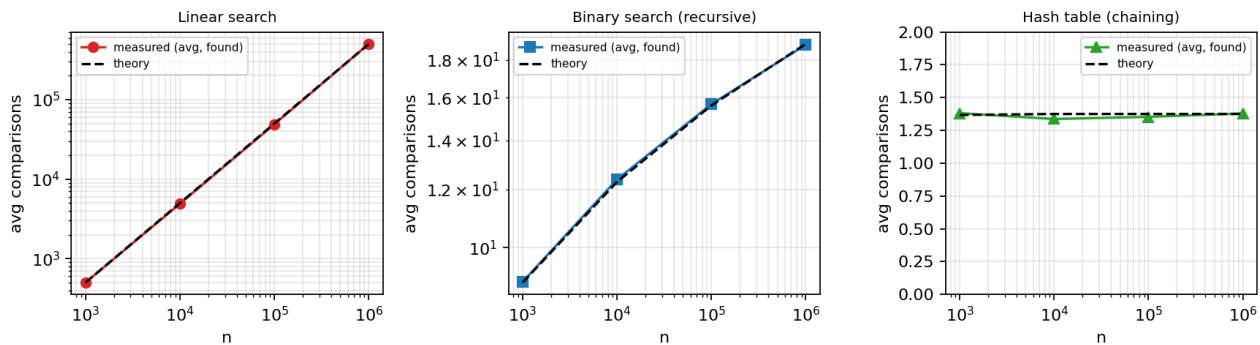
Reg No: 192425147 | D.Hasini

Figure 9.7: Preprocessing time vs n (log-log axes)

Task 10 – Compare Theory with Experimental Behaviour

Comparisons agree almost exactly with the derivations. Figure 10.1 plots measured average comparisons against the formulas of Task 3. Linear search needs 502,907 comparisons at $n = 10^6$ against the predicted 500,000.5 (difference 0.58%, the sampling error of only 100 random queries). Binary search needs 18.88 against the predicted 18.93, and its worst case equals $\lceil \log_2 n \rceil + 1$ exactly at every size. The hash table needs 1.38 against $1 + \alpha/2 = 1.375$; the small wiggles in Figure 10.1 (right) are on a magnified axis (1.34–1.38) and are sampling noise of 1,000 queries.

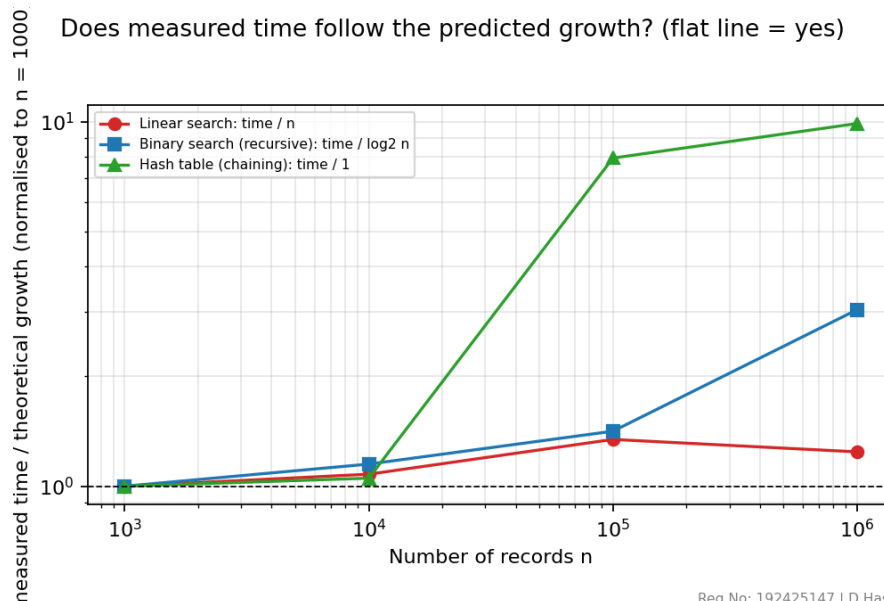
Theoretical vs measured comparisons (successful search)



Reg No: 192425147 | D.Hasini

Figure 10.1: Theoretical (dashed) versus measured average comparisons

Execution time follows the same growth order, but not perfectly. Figure 10.2 divides measured time by the theoretical growth function (n , $\log_2 n$, 1) and normalises to $n = 1,000$; a flat line at 1 would mean exact agreement.



Reg No: 192425147 | D.Hasini

Figure 10.2: Measured time divided by predicted growth (normalised to $n = 1,000$)

Which scales better? The hash table has the flattest growth, binary search is next, and linear search is far behind: at $n = 10^6$ one linear search costs 1,788× one binary search and 7,779× one hash lookup.

Differences between prediction and measurement, and likely reasons.

Observation	Possible reasons
Binary-search time grew 6.1× from 10^3 to 10^6 records, more than the 2× predicted by $\log n$.	Cache effects: a 1,000-record list sits entirely in the CPU cache, while a 10^6 -record list does not, so the later comparisons of each search are cache misses. Python call overhead per recursive level (about 20 calls) also has a constant cost not counted in the comparison model.

Observation	Possible reasons
Hash-search time grew 9.9× although comparisons stayed at about 1.4.	The comparison count is constant but each lookup touches the bucket array, a list object and a tuple at random memory addresses; at 10^6 records (hundreds of MB of objects) these accesses miss the cache. So the real cost of 'O(1)' depends on the memory hierarchy, which the model ignores.
Linear time per comparison was almost constant (about 23 ns at $n = 10^6$, 19 ns at $n = 10^3$).	As predicted: sequential scanning is cache-friendly (prefetching), so linear time $\propto n$ holds very well.
Run-to-run timing varies, comparison counts do not.	Timing is affected by CPU scheduling, frequency scaling and garbage collection; comparison counts are deterministic for a given seed. This is why I validate the <i>theory</i> mainly with counts and use times for the overall trend.
Hash build is the slowest preprocessing (and sorting the fastest) although both are 'n log n' or 'n' in theory.	Sorting is performed by Timsort in compiled C code, whereas my hash-table build is an interpreted Python loop with object allocation for each entry. The constant factor, not the growth rate, differs.
Conclusion of the comparison. The mathematical analysis predicts the <i>growth</i> and the ranking of the algorithms correctly; absolute times differ from simple operation counts because of constant factors, memory hierarchy and implementation language.	

Task 11 — Recommend and Defend the Final Design

Recommendation: binary search on a sorted array (recursive or iterative), with the records sorted once as preprocessing.

Why it is more appropriate than the alternatives

- **Versus linear search.** At $n = 10^6$ a linear search needs about 502,907 comparisons and 11.8 ms, binary search needs 18.9 comparisons and 6.6 μ s — about 1,788 $\times$ faster. The one-time sort (258 ms) is repaid after only about 15 searches (Task 12), which a university system exceeds within minutes.

- **Versus the hash table.** The hash table is faster per search (1.5 μ s vs 6.6 μ s) but its build costs 8 $\times$ more (2.1 s vs 0.26 s), it needs about 13 $\times$ more extra memory (153 MB vs 12 MB at 10^6 records), its worst case is $O(n)$ rather than $O(\log n)$, and it can only answer exact-match queries. A sorted array also answers range questions the staff are likely to ask (for example ‘all students whose registration numbers lie between X and Y’) in $O(\log n + k)$.

Criterion ($n = 10^6$)	Linear	Binary (recommended)	Hash
Theoretical search cost	$\Theta(n)$	$\Theta(\log n)$	$\Theta(1)$ expected
Measured avg comparisons (found)	502,907.31	18.88	1.38
Measured avg time per search (μ s)	11,816.50	6.61	1.52
Preprocessing time (ms)	0	258.49	2,083.93
Extra memory	none	12 MB	153 MB
Worst case	n	20	9 measured, $O(n)$ theoretical
Range / ordered queries	yes (slow)	yes (fast)	no

Table 11.1: Evidence used for the recommendation (from my own experiments and derivations).

Trade-offs and limitations. (1) The array must stay sorted: inserting one record into a sorted array costs $O(n)$, so new records should be added in batches and merged (Task 12). (2) The sort is an up-front cost of $\Theta(n \log n)$ — irrelevant when the search count is large, but not worth it for a handful of one-off lookups, where linear search is simpler. (3) A hash table would be a better choice only if exact-match lookups are by far the dominant operation and memory is plentiful.

Does it stay practical as the database grows? Yes. Because $T(n) = \Theta(\log n)$, going from 10^6 to 10^9 records adds only about 10 comparisons per search ($20 \rightarrow 30$), and the sort cost grows only slightly faster than linearly. Linear search would be 1,000 times slower at 10^9 records.

Task 12 — Day-4 Design Challenge: Re-evaluate and Adapt the Design

Changed requirement: 50,000 search requests per hour on 1,000,000 records; new records are added only once per day.

A. Impact Analysis (2 marks)

What has changed. The original problem asked how to find a registration number efficiently as the database grows; the workload (how many searches) was not specified. Now it is: 50,000 searches per hour = 13.9 searches per second = **1,200,000 searches per day** — more searches per day than there are records. At the same time the data change only once per day.

Original assumptions that are affected. (i) In Tasks 3–11 preprocessing was regarded as an extra cost that must be justified against a few searches; with 1.2 million searches a day it is divided over all of them (sorting 10^6 records costs only about $0.15 \mu\text{s}$ per search per day). (ii) Per-search cost, which looked small in isolation, is now multiplied by 1.2 million, so even a small difference per search matters. (iii) The assumption that records might be inserted at any time (which makes a sorted array or hash table awkward to maintain) is no longer needed.

Why the search frequency is important. Total cost = preprocessing + (number of searches $\times$ cost per search). When the number of searches is huge the second term dominates, so the algorithm with the cheapest search is preferable *unless* its preprocessing or memory is unaffordable.

Why rare updates matter. If records are added only once a day, the index (sorted array or hash table) can be rebuilt or merged **once a night** and then stays read-only for 24 hours: no per-insertion cost, no locking and no re-balancing during the day. A static structure is exactly what binary search and a hash table need; the weak point of both (expensive insertion) is irrelevant.

B. Evaluation of the Original Design (2 marks)

Original recommendation: binary search on a sorted array. The benchmark (src/task12_benchmark.py, 10^6 records, 50,000 searches, ~90% existing and ~10% missing keys, including my own reg no and reg no + 1) gives:

Design	Preprocessing (s)	50,000 searches (s)	Per search (μs)	Max searches/s (1 core)	Extra memory (MB)	Cost of one day (s)
Linear (projected from 100 measured searches)	0	575.95	11,519.08	87	0	13,823
Binary search (original)	0.18	0.18	3.62	276,091	12	4.52
Hash table	1.19	0.03	0.68	1,474,926	153	2.00

Table 12.1: Benchmark of the three designs for 50,000 searches on 1,000,000 records. 'Cost of one day' = preprocessing + 24 hours $\times$ (50,000-search time).

```

Command Prompt
C:\Users\hasin\OneDrive\Desktop\DAA-A1-192425147-D.Hasini\src>python task12_benchmark.py
Reg No: 192425147 | Name: D.Hasini | Seed: 5147 | 2026-10-06 23:00:14
TASK 12 - Re-evaluation benchmark: 50,000 searches on 1,000,000 records
=====
A. Preprocessing (done once per day)
sort for binary search : 0.175 s extra memory ~ 12.0 MB (second list of references)
hash-table build : 1.190 s extra memory ~ 152.6 MB (m = 1,333,357 buckets)

B. Serving 50,000 searches (mix: ~90% existing keys, ~10% missing; includes own reg no + reg no+1)
design prep (s) search total (s) per search (us) prep+search (s) searches/s
Linear search (PROJECTED from 100) 0.000 575.954 11519.08 575.954 87
Binary search on sorted array 0.175 0.181 3.62 0.356 276,138
Hash table (chaining) 1.190 0.034 0.68 1.224 1,476,852
correctness: hits found by binary = 45,058, hash = 45,058, expected = 45,058 -> MATCH
required rate = 50,000/hour = 13.89 searches/second
Linear search (PROJECTED from 100) capacity 87/s on one core -> meets the requirement (CPU busy 15.999% of one core)
Binary search on sorted array capacity 276,138/s on one core -> meets the requirement (CPU busy 0.005% of one core)
Hash table (chaining) capacity 1,476,852/s on one core -> meets the requirement (CPU busy 0.001% of one core)

C. Nightly update with 4,995 new records (once per day)
re-sort everything : 0.195 s
merge sorted batch (O(n+k)) : 0.114 s
rebuild hash index : 1.027 s

D. Break-even against linear search (searches needed to repay preprocessing)
binary search : 15.2 searches
hash table : 103.3 searches
searches per day at 50,000/hour = 1,200,000 (preprocessing happens once per day)
daily preprocessing cost per search: binary = 0.15 us, hash = 0.99 us

results saved to results/task12_results.csv
C:\Users\hasin\OneDrive\Desktop\DAA-A1-192425147-D.Hasini\src>

```

Figure 12.1: Terminal output of the Task 12 benchmark (50,000 searches on 1,000,000 records)

Task 12: cost of 50,000 searches on 1,000,000 records

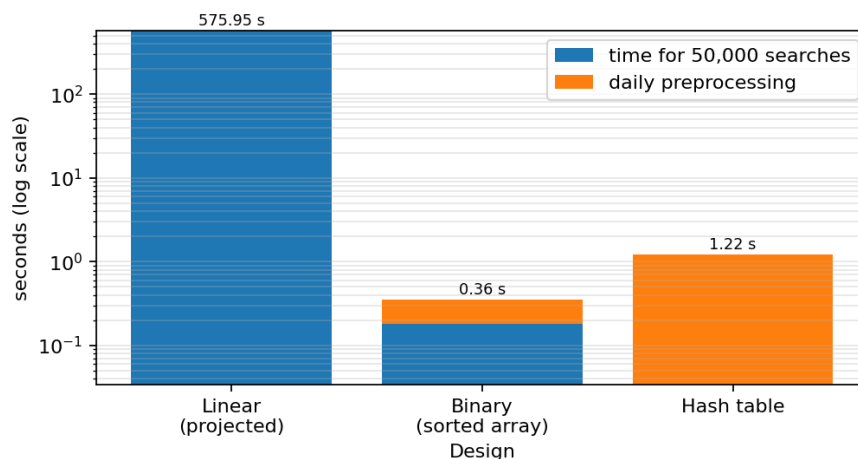


Figure 12.2: Total time for 50,000 searches including the daily preprocessing (log scale)

Behaviour under 50,000 searches/hour. The required rate is only 13.9 searches/second. Binary search can sustain about 276,091 per second on one core, i.e. it would be busy 0.005% of the time. **Linear search** would, on paper, also meet the rate (87/s > 13.9/s), but only by using about 16% of a CPU core continuously, each request taking about 12 ms. It would have no headroom: a database or request rate about 6× larger would saturate the core completely. It is therefore not acceptable.

Is the preprocessing cost acceptable? Yes: sorting takes 0.18 s once a day. The hash-table build (1.2 s) is also acceptable in absolute terms, but it is 7× slower than the sort and needs 153 MB instead of 12 MB.

Could another approach give better overall performance? Yes, marginally: the hash table is the cheapest design per day (2.0 s of CPU versus 4.5 s for binary search), because its faster searches (0.7 μs vs 3.6 μs) outweigh its larger build over 1.2 million searches. But the saving is only about 2.5 seconds out of 86,400 seconds in a day (about 0.003% of one core).

C. Adaptation or Redesign (2 marks)

Decision: MODIFY — keep the search strategy (binary search on a sorted array, $\Theta(\log n)$ per search) but modify how the sorted array is prepared and maintained.

Aspect	Original design	Revised design
Search	recursive binary search	same algorithm; iterative form may be used in production to avoid recursion overhead (same $\Theta(\log n)$)
When the data are prepared	sort the complete data set whenever it changes ($\Theta(n \log n)$)	nightly batch job : sort only the day's k new records ($\Theta(k \log k)$) and merge them into the existing sorted array ($\Theta(n + k)$)
During the day	records could change at any time	the sorted array is an immutable snapshot , read-only for 24 h; the new snapshot replaces it atomically after the merge
Cost of the daily update	re-sort of $10^6 + k$ records: 0.20 s measured	merge of the sorted batch: 0.11 s measured ($k \approx 5,000$ new records)

Effect on computational complexity. Searching is unchanged: $\Theta(\log n)$, about 20 comparisons for $n = 10^6$. Daily preparation improves from $\Theta(n \log n)$ to $\Theta(n + k \log k)$, where k is the number of new records. Total cost of one day: $\Theta(n + k \log k + q \log n)$ with $q = 1,200,000$ searches, i.e. about $10^6 + 6 \times 10^4 + 2.4 \times 10^7$ comparisons. In my measurement the merge took 0.11 s against 0.20 s for a complete re-sort; the gain is small in Python because Timsort already exploits existing order, but the asymptotic advantage grows with n . For comparison, rebuilding a hash table after the daily update took 1.0 s.

D. Evidence-Based Defence (2 marks)

Evidence	Finding	Consequence for the decision
Search frequency	13.9 searches/s = 1.2 M per day	linear search (16% of a core, ~12 ms per request, no headroom) is rejected; an indexed structure is needed
Dataset size	10^6 records; $\log_2 n \approx 20$	binary search needs about 19 comparisons per search
Update frequency	once per day	static structure + nightly merge is enough; no online insertion needed
Preprocessing cost	sort 0.18 s, merge 0.11 s, hash build 1.2 s	binary's preprocessing is the cheapest, 7× less than hash
Capacity	binary 276,091/s vs need 13.9/s	binary has about 19,879× headroom; the hash table's extra speed (0.7 vs 3.6 μ s) adds no real benefit
Memory	12 MB vs 153 MB	binary search is far lighter
Functionality	sorted order	range queries (a block of registration numbers) still possible; hash would lose them

Final decision. Retain binary search as the search algorithm and modify the preprocessing into a nightly sorted-batch merge that produces a read-only snapshot. This design serves 50,000 searches in 0.18 s (about 0.0050% of the hour), prepares the daily data in about 0.11 s, uses 12 MB of extra memory and keeps range queries. The hash table remains the fastest option per search and costs 2.5 s less CPU per day, so it should be reconsidered only if the request rate grew by several orders of magnitude (then the extra 141 MB and the 1.2-second nightly build would be worth paying). With the given requirement, the evidence does not justify that complexity.

Limitations of this study. Timings are from one machine and one run per size in Python; the linear-search time for 50,000 searches is projected from 100 measured searches, not measured in full; the new-records-per-day figure (about 5,000) is an assumption. The conclusions depend on the ranking and ratios of the measurements, which are supported by the exact comparison counts.